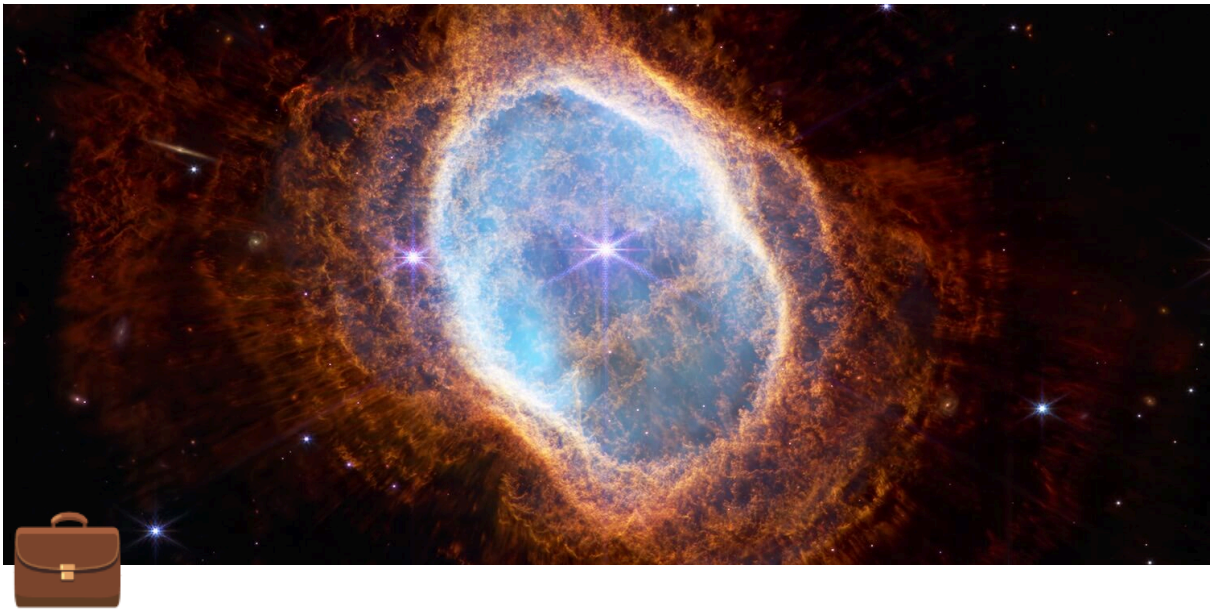

Miniprojet 2 — Data Lake & Analyse du Marché de l'Emploi IT au Maroc

Mexora RH Intelligence — Construire un Data Lake pour éclairer la stratégie de recrutement

> **NB** : à rendre avant le 1er Juin 2026,
> **Livrable** : *Rapport PDF (incluant repository Github)*,
> **Plateforme** : ClassRoom (*ne pas envoyer vos travaux par email*),
> **Important** : La date de livraison est cruciale et est comptée dans l'évaluation du travail !

📢 Indication

> Les scripts Python et SQL fournis dans cet énoncé sont donnés **à titre indicatif**. Vous devez vous assurer de leur bon fonctionnement avant de les utiliser.
> Les fichiers de données sont **à générer par vos soins**, en respectant les structures et les problèmes intentionnels décrits dans chaque section. Aucun fichier ne sera fourni.
> Vous êtes **libres** de formuler vos propres hypothèses et choix de conception,

> à condition qu'ils soient cohérents avec l'énoncé et explicitement justifiés dans votre rapport. Un choix assumé et argumenté sera toujours valorisé, même s'il diffère de la solution attendue.

🎯 Contexte et mise en situation

Mexora croît rapidement. Après deux années d'expansion commerciale, le Directeur des Ressources Humaines constate que l'équipe data est sous-dimensionnée : 2 personnes pour gérer tout le système analytique d'une marketplace traitant 50 000 commandes par mois.

La direction a validé le recrutement de **5 nouveaux profils data** dans les 6 prochains mois — Data Engineers, Data Analysts et un Data Scientist. Mais avant de publier les offres et de fixer les salaires, le DRH formule une demande précise :

> *"Je veux comprendre le marché marocain de l'emploi IT avant de recruter. Quelles compétences sont les plus demandées ? Quel est le salaire médian à Tanger pour un Data Engineer ? Est-ce que les entreprises recrutent plutôt en CDI ou en freelance ? Qui sont nos concurrents sur le marché du talent ?"*

Votre mission : **construire un Data Lake**, le peupler avec des données réelles d'offres d'emploi IT marocaines, le transformer en zones Bronze/Silver/Gold, et produire une **analyse stratégique du marché** qui guidera concrètement la politique RH de Mexora.

📋 Objectifs pédagogiques

À la fin de ce miniprojet, vous serez capable de :

- Concevoir une **architecture Data Lake** en zones Bronze / Silver / Gold
- Justifier les **choix de formats** de stockage (JSON, CSV, Parquet)
- Développer des **pipelines de traitement Python** sur des données semi-structurées
- Extraire des informations depuis du **texte libre** (NLP basique, regex)
- Utiliser **DuckDB** pour l'analytique SQL sur fichiers locaux
- Produire une **analyse de marché** avec insights stratégiques actionnables
- Rédiger un **rapport professionnel** à destination d'une direction non-technique

📦 Données fournies

Dataset principal — `offres_emploi_it_maroc.json`

5 000 offres d'emploi IT marocaines collectées sur Rekrute, MarocAnnonce et LinkedIn Maroc entre janvier 2023 et novembre 2024. Les données sont brutes, telles que récupérées par scraping — imparfaites par nature.

Structure d'une offre :

```
{
  "id_offre": "RK-2024-08471",
  "source": "rekrute",
  "titre_poste": "Développeur Full Stack React/Node.js",
  "description": "Nous recherchons un développeur Full Stac
k expérimenté maîtrisant React, Node.js, PostgreSQL. Connai
ssance de Docker et AWS appréciée. Le candidat devra travai
ller en méthode Agile avec une équipe de 5 personnes...",
  "competences_brut": "React, Node.js, PostgreSQL, Docker,
AWS, Agile, Git",
  "entreprise": "TechMaroc SARL",
  "ville": "Casablanca",
  "type_contrat": "CDI",
  "experience_requise": "3-5 ans",
  "salaire_brut": "15000-20000 MAD",
  "niveau_etudes": "Bac+5",
  "secteur": "Informatique / Télécom",
  "date_publication": "2024-08-15",
  "date_expiration": "2024-09-15",
  "nb_postes": 1,
  "teletravail": "Hybride",
  "langue_requise": ["Français", "Anglais"]
}
```

Problèmes connus dans les données brutes :

Champ	Problèmes à traiter
`titre_poste`	Intitulés non standardisés : "Dev Data", "Ingénieur Big Data", "Data Eng.", "Développeur BI", "Data Engineer Junior" → tous à

Champ	Problèmes à traiter
	normaliser
`competences_brut`	Texte libre mélangé avec la description, séparateurs incohérents (virgule, `/`, `•`, retours à la ligne)
`salaire_brut`	Formats mixtes : `"15000-20000 MAD"`, `"15K-20K"`, `"Selon profil"`, `null`, `"Confidentiel"`, montants en EUR à convertir
`ville`	Mêmes incohérences que le miniprojet 1 : `"casa"`, `"CASABLANCA"`, `"Casablanca"`
`experience_requise`	Formats variés : `"3-5 ans"`, `"3 à 5 ans"`, `"min 3 ans"`, `"Débutant accepté"`, `null`
`type_contrat`	`"CDI"`, `"cdi"`, `"Contrat à durée indéterminée"`, `"Permanent"`
`date_publication`	Formats mixtes, certaines dates incohérentes (publication > expiration)
`description`	Champ non structuré — source principale pour l'extraction de compétences

Fichier complémentaire — `referentiel_competences_it.json`

Dictionnaire de **300 compétences IT** organisées en familles, fourni comme référence pour la normalisation :

```
{
  "familles": {
    "langages": {
      "python": ["python", "python3", "py"],
      "javascript": ["javascript", "js", "node.js", "nodej
s", "node"],
      "java": ["java", "java8", "java11", "java17"],
      "sql": ["sql", "mysql", "postgresql", "postgres", "or
acle", "tsql"],
      "r": ["r", "rlang", "r-studio"]
    },
    "frameworks_web": {
      "react": ["react", "reactjs", "react.js"],
      "angular": ["angular", "angularjs"],
      "django": ["django", "django rest"],
      "spring": ["spring", "spring boot", "springboot"]
    },
```

```
    "data_engineering": {
      "spark": ["spark", "apache spark", "pyspark"],
      "kafka": ["kafka", "apache kafka"],
      "airflow": ["airflow", "apache airflow"],
      "dbt": ["dbt", "data build tool"],
      "hadoop": ["hadoop", "hdfs", "mapreduce"]
    },
    "cloud": {
      "aws": ["aws", "amazon web services", "ec2", "s3", "l
ambda"],
      "gcp": ["gcp", "google cloud", "bigquery", "cloud sto
rage"],
      "azure": ["azure", "microsoft azure", "synapse"]
    },
    "bi_analytics": {
      "power_bi": ["power bi", "powerbi", "pbi"],
      "tableau": ["tableau", "tableau desktop"],
      "metabase": ["metabase"],
      "looker": ["looker", "looker studio"]
    }
  }
}
```

Fichier complémentaire — `entreprises_it_maroc.csv`

```
nom_entreprise, secteur, taille (PME/ETI/Grande Entreprise/Startup),
ville_siege, site_web, type (SSII/Produit/Conseil/Telecom/Banque/Autre)
```

🏗️ Architecture Data Lake à concevoir

Vous devez implémenter l'architecture suivante en justifiant chaque choix :

```
offres_emploi_it_maroc.json  (données brutes)
            |
            ▼
┌──────────────────────────────────────────┐
|  ZONE BRONZE  (Raw — données brutes)      |
|  Format : JSON (préservé tel quel)        |
|  Accès : archive immuable, jamais modifiée |
```

```
│  Partitionnement : par source et par mois    │
└──────────────────────────────────────────────┘
                      │  Nettoyage + Standardisation
                      ▼
┌──────────────────────────────────────────────┐
│  ZONE SILVER  (Cleaned — données nettoyées)  │
│  Format : Parquet (compressé, performant)    │
│  Contenu : offres propres, champs typés      │
│  Partitionnement : par ville et par mois     │
└──────────────────────────────────────────────┘
                      │  Agrégation + Enrichissement
                      ▼
┌──────────────────────────────────────────────┐
│  ZONE GOLD  (Curated — données analytiques)  │
│  Format : Parquet (tables analytiques)       │
│  Contenu : agrégats, KPIs, indicateurs       │
│  Usage : requêtes DuckDB, dashboard, rapport │
└──────────────────────────────────────────────┘
```

Structure de répertoires à créer

```
data_lake_mexora_rh/
├── bronze/
│   ├── rekrute/
│   │   ├── 2024_01/offres_raw.json
│   │   ├── 2024_02/offres_raw.json
│   │   └── ...
│   ├── marocannonce/
│   │   └── ...
│   └── linkedin/
│       └── ...
├── silver/
│   ├── offres_clean/
│   │   └── offres_clean.parquet
│   └── competences_extraites/
│       └── competences.parquet
└── gold/
    ├── top_competences.parquet
    ├── salaires_par_profil.parquet
    ├── offres_par_ville.parquet
    ├── entreprises_recruteurs.parquet
    └── tendances_mensuelles.parquet
```

🏗️ Travail demandé

Étape 1 — Architecture Data Lake (20%)

1.1 Document de conception

Produire un document de conception de l'architecture (2 pages maximum) répondant aux questions suivantes :

Justification des formats :

Zone	Format choisi	Pourquoi ce format ?	Pourquoi pas les autres ?
Bronze	JSON	À justifier	À justifier
Silver	Parquet	À justifier	À justifier
Gold	Parquet	À justifier	À justifier

Questions de conception à traiter :

1. Pourquoi conserver les données brutes en zone Bronze sans les modifier ? Quels risques si on ne le fait pas ?
2. Qu'est-ce que le "schema-on-read" ? En quoi est-ce différent du DWH du miniprojet 1 ("schema-on-write") ?
3. Comment définit-on la partition dans chaque zone ? Justifier le choix `par_source/par_mois` en Bronze et `par_ville/par_mois` en Silver.
4. Comment éviter que le Data Lake devienne un "Data Swamp" ? Quelles règles de gouvernance mettre en place ?

1.2 Schéma d'architecture

Produire un schéma visuel de l'architecture (draw.io, Lucidchart, ou équivalent) montrant :
- Les 3 zones avec leurs formats
- Les flux de données entre zones
- Les outils utilisés à chaque étape
- Les utilisateurs finaux (qui consomme quelle zone ?)

Livrable Étape 1

- Document de conception (PDF, 2 pages max)
- Schéma d'architecture (image)
- Structure de répertoires créée sur votre machine (capture d'écran)

Étape 2 — Pipeline de traitement Python (40%)

2.1 Structure du projet

```
mexora_rh_lake/
├── pipeline/
│   ├── bronze_ingestion.py      # Chargement brut dans la zone Bronze
│   ├── silver_transform.py      # Nettoyage et standardisation → Silver
│   ├── silver_nlp.py            # Extraction de compétences depuis texte
│   ├── gold_aggregation.py      # Calcul des agrégats → Gold
│   └── utils.py                 # Fonctions partagées
├── analysis/
│   └── analyse_marche.py        # Requêtes DuckDB analytiques
├── data_lake/                   # Répertoire racine du Data Lake
├── main.py                      # Orchestration
└── requirements.txt
```

2.2 Bronze → Ingestion brute

```python
import json
import os
from datetime import datetime

def ingerer_bronze(filepath_source: str, data_lake_root: st
r) -> dict:
    """
    Charge les données brutes dans la zone Bronze sans aucu
ne modification.
    Partitionne par source et par mois de publication.

    Principe fondamental : la zone Bronze est IMMUABLE.
    On ne modifie JAMAIS les données une fois chargées en B
ronze.
    C'est l'archive fidèle de ce qui a été reçu.
    """
    with open(filepath_source, 'r', encoding='utf-8') as f:
        data = json.load(f)

    offres = data.get('offres', [])
    stats = {'total': len(offres), 'par_source': {}, 'par_m
ois': {}}

    # Partitionnement par source et par mois
    partitions = {}
    for offre in offres:
```

```
        source = offre.get('source', 'inconnu').lower().rep
lace(' ', '_')
        date_pub = offre.get('date_publication', '')

        try:
            mois_partition = datetime.strptime(date_pub[:
7], '%Y-%m').strftime('%Y_%m')
        except (ValueError, TypeError):
            mois_partition = 'date_inconnue'

        cle = f"{source}/{mois_partition}"
        if cle not in partitions:
            partitions[cle] = []
        partitions[cle].append(offre)

    # Écriture dans Bronze
    nb_fichiers = 0
    for partition, offres_partition in partitions.items():
        chemin_dir = os.path.join(data_lake_root, 'bronze',
partition)
        os.makedirs(chemin_dir, exist_ok=True)

        chemin_fichier = os.path.join(chemin_dir, 'offres_r
aw.json')
        with open(chemin_fichier, 'w', encoding='utf-8') as
f:
            json.dump({
                'metadata': {
                    'source_fichier': filepath_source,
                    'date_ingestion': datetime.now().isofor
mat(),
                    'partition': partition,
                    'nb_offres': len(offres_partition)
                },
                'offres': offres_partition
            }, f, ensure_ascii=False, indent=2)

        nb_fichiers += 1
```

```
        source_nom = partition.split('/')[0]
        stats['par_source'][source_nom] = stats['par_sourc
e'].get(source_nom, 0) + len(offres_partition)

    print(f"[BRONZE]{stats['total']} offres ingérées dans{n
b_fichiers} partitions")
    return stats
```

2.3 Bronze → Silver : Nettoyage et standardisation

```
import pandas as pd
import re
import json
from pathlib import Path

def charger_depuis_bronze(data_lake_root: str) -> pd.DataFr
ame:
    """Charge et consolide toutes les offres depuis la zone
Bronze."""
    all_offres = []
    bronze_path = Path(data_lake_root) / 'bronze'

    for json_file in bronze_path.rglob('offres_raw.json'):
        with open(json_file, 'r', encoding='utf-8') as f:
            data = json.load(f)
        all_offres.extend(data.get('offres', []))

    df = pd.DataFrame(all_offres)
    print(f"[SILVER]{len(df)} offres chargées depuis Bronz
e")
    return df


def nettoyer_titres_postes(df: pd.DataFrame) -> pd.DataFram
e:
    """
    Standardise les intitulés de poste en familles de profi
ls IT.
```

```
    Règle métier : un titre est normalisé vers le profil le
plus proche.
    Les titres non reconnus sont conservés tels quels avec
flag 'non_classifié'.
    """
    mapping_profils = {
        # Data Engineering
        r'data\s*eng(ineer|ineer\w*|\.)?|ingénieur\s+data|d
ev\s+data\s+eng': 'Data Engineer',
        r'etl\s*dev|pipeline\s*dev|ingénieur\s+etl':
'Data Engineer',

        # Data Analysis
        r'data\s*anal(yst|yste|ytics)|analyste?\s+data|bi\s
+anal':            'Data Analyst',
        r'business\s+intel(ligence)?|ingénieur\s+bi|dévelop
peur\s+bi':        'Data Analyst',
        r'reporting\s+(anal|spec|officer)':
'Data Analyst',

        # Data Science
        r'data\s*sci(entist|ence)|machine\s*learn|ml\s*eng|
ia\s*eng':         'Data Scientist',
        r'deep\s*learn|nlp\s*eng|computer\s*vision':
'Data Scientist',

        # Software Engineering
        r'full\s*stack|fullstack':
'Développeur Full Stack',
        r'back[\s-]*end|backend':
'Développeur Backend',
        r'front[\s-]*end|frontend':
'Développeur Frontend',
        r'dev(eloppeur|eloper)?\s+mobile|ios\s+dev|android
\s+dev':            'Développeur Mobile',

        # Infrastructure
```

```
        r'devops|sre|site\s*reliab':
'DevOps / SRE',
        r'cloud\s*(arch|eng|admin)|aws\s+eng|gcp\s+eng|azur
e\s+eng':         'Cloud Engineer',
        r'sys(admin|tème)|réseau\s+inf|network\s+eng':
'Admin Systèmes & Réseaux',

        # Cyber
        r'cyber|sécurité\s+info|pentester|soc\s+anal':
'Cybersécurité',

        # Management
        r'chef\s+de\s+proj(et)?|project\s+man|scrum\s*maste
r':               'Chef de Projet IT',
        r'architect(e)?\s+(log|tech|data|cloud|sol)':
'Architecte IT',
    }

    df['profil_normalise'] = 'Autre IT'
    df['profil_source'] = df['titre_poste'].str.lower().st
r.strip()

    for pattern, profil in mapping_profils.items():
        masque = df['profil_source'].str.contains(pattern,
regex=True, na=False)
        df.loc[masque, 'profil_normalise'] = profil

    non_classes = (df['profil_normalise'] == 'Autre IT').su
m()
    print(f"[SILVER] Titres :{non_classes} offres classées
'Autre IT' sur{len(df)}")
    return df


def normaliser_salaires(df: pd.DataFrame) -> pd.DataFrame:
    """
    Extrait et normalise les salaires en MAD mensuel brut.
```

```
    Règles :
      - Fourchettes → extraire min et max, calculer médiane
      - "K" → multiplier par 1000
      - EUR → convertir en MAD (taux fixe 1 EUR = 10.8 MAD
pour 2024)
      - "Selon profil", "Confidentiel", null → salaire_conn
u = False
    """
    TAUX_EUR_MAD = 10.8

    def parser_salaire(valeur):
        if pd.isna(valeur) or str(valeur).lower() in ['nul
l', 'confidentiel', 'selon profil', '']:
            return None, None, False

        s = str(valeur).lower().replace(' ', '').replace
('\u202f', '')

        # Conversion EUR → MAD
        est_eur = 'eur' in s or '€' in s
        s = s.replace('eur', '').replace('€', '').replace
('mad', '').replace('dh', '')

        # Gestion "K" (milliers)
        s = re.sub(r'(\d+(?:\.\d+)?)k', lambda m: str(int(f
loat(m.group(1)) * 1000)), s)

        # Extraction des montants
        nombres = re.findall(r'\d+(?:\.\d+)?', s)

        if not nombres:
            return None, None, False

        montants = [float(n) for n in nombres]

        if est_eur:
            montants = [m * TAUX_EUR_MAD for m in montants]
```

```
        if len(montants) >= 2:
            sal_min = min(montants[:2])
            sal_max = max(montants[:2])
        else:
            sal_min = sal_max = montants[0]

        # Cohérence : salaires IT Maroc entre 3 000 et 100
000 MAD
        if sal_min < 3000 or sal_max > 100000:
            return None, None, False

        return sal_min, sal_max, True

    resultats = df['salaire_brut'].apply(lambda x: pd.Serie
s(parser_salaire(x),
                                          index=['salaire_m
in_mad', 'salaire_max_mad', 'salaire_connu']))
    df = pd.concat([df, resultats], axis=1)
    df['salaire_median_mad'] = (df['salaire_min_mad'] + df
['salaire_max_mad']) / 2

    pct_connu = df['salaire_connu'].mean() * 100
    print(f"[SILVER] Salaires :{pct_connu:.1f}% des offres
ont un salaire renseigné et valide")
    return df


def normaliser_experience(df: pd.DataFrame) -> pd.DataFram
e:
    """
    Transforme l'expérience en valeur numérique (années min
requises).

    Exemples :
      "3-5 ans"          → experience_min = 3, experience_m
ax = 5
      "min 3 ans"        → experience_min = 3, experience_m
ax = None
```

```
      "Débutant accepté" → experience_min = 0, experience_m
ax = 2
      "Senior (7+ ans)"  → experience_min = 7, experience_m
ax = None
    """
    def parser_experience(valeur):
        if pd.isna(valeur):
            return None, None

        s = str(valeur).lower()

        if any(mot in s for mot in ['débutant', 'junior',
'stage', 'sans expérience']):
            return 0, 2
        if any(mot in s for mot in ['senior', 'confirmé',
'expert', 'lead']):
            return 5, None

        fourchette = re.search(r'(\d+)\s*[-àa]\s*(\d+)', s)
        if fourchette:
            return int(fourchette.group(1)), int(fourchett
e.group(2))

        min_seul = re.search(r'(\d+)\s*(?:ans?|years?)', s)
        if min_seul:
            return int(min_seul.group(1)), None

        return None, None

    resultats = df['experience_requise'].apply(
        lambda x: pd.Series(parser_experience(x), index=['e
xperience_min_ans', 'experience_max_ans'])
    )
    df = pd.concat([df, resultats], axis=1)
    return df
```

2.4 Silver → Silver : Extraction des compétences (NLP basique)

C'est la transformation la plus importante et la plus originale de ce projet. Elle transforme du texte libre en données structurées.

```
def extraire_competences(df: pd.DataFrame, referentiel_pat
h: str) -> pd.DataFrame:
    """
    Extrait les compétences IT depuis deux sources :
      1. Le champ 'competences_brut' (liste semi-structuré
e)
      2. Le champ 'description' (texte libre — plus riche m
ais plus bruité)

    Stratégie : matching sur le référentiel de compétences
normalisé.
    Chaque offre produit une ligne par compétence détectée.
    """
    with open(referentiel_path, 'r', encoding='utf-8') as
f:
        referentiel = json.load(f)

    # Construire un dictionnaire plat : alias → nom_normali
se, famille
    dict_competences = {}
    for famille, competences in referentiel['familles'].ite
ms():
        for nom_normalise, aliases in competences.items():
            for alias in aliases:
                dict_competences[alias.lower()] = {
                    'competence': nom_normalise,
                    'famille': famille
                }

    # Trier par longueur décroissante pour éviter les faux
positifs
    # (ex: "node" ne doit pas matcher avant "node.js")
    aliases_tries = sorted(dict_competences.keys(), key=le
n, reverse=True)
```

```
    resultats = []

    for _, offre in df.iterrows():
        # Concaténer les deux sources de texte
        texte_complet = ' '.join(filter(None, [
            str(offre.get('competences_brut', '') or ''),
            str(offre.get('description', '') or '')
        ])).lower()

        competences_trouvees = set()

        for alias in aliases_tries:
            # Recherche en tant que mot entier (word bounda
ry)
            pattern = r'\b' + re.escape(alias) + r'\b'
            if re.search(pattern, texte_complet):
                info = dict_competences[alias]
                cle = info['competence']
                if cle not in competences_trouvees:
                    competences_trouvees.add(cle)
                    resultats.append({
                        'id_offre':      offre['id_offre'],
                        'profil':        offre.get('profil_
normalise'),
                        'ville':         offre.get('ville_s
td'),
                        'competence':    info['competenc
e'],
                        'famille':       info['famille'],
                        'date_pub':      offre.get('date_pu
blication'),
                        'annee':         str(offre.get('dat
e_publication', ''))[:4],
                        'mois':          str(offre.get('dat
e_publication', ''))[5:7],
                    })

        # Offre sans compétence détectée : tracer
```

```
        if not competences_trouvees:
            resultats.append({
                'id_offre':      offre['id_offre'],
                'profil':        offre.get('profil_normalis
e'),
                'ville':         offre.get('ville_std'),
                'competence':    'non_détecté',
                'famille':       'inconnu',
                'date_pub':      offre.get('date_publicatio
n'),
                'annee':         str(offre.get('date_public
ation', ''))[:4],
                'mois':          str(offre.get('date_public
ation', ''))[5:7],
            })

    df_competences = pd.DataFrame(resultats)

    nb_offres_avec = df_competences[df_competences['compete
nce'] != 'non_détecté']['id_offre'].nunique()
    print(f"[SILVER NLP]{len(df_competences)} lignes compét
ences extraites")
    print(f"[SILVER NLP]{nb_offres_avec}/{len(df)} offres o
nt au moins 1 compétence détectée")

    return df_competences


def sauvegarder_silver(df_offres: pd.DataFrame, df_competen
ces: pd.DataFrame,
                       data_lake_root: str):
    """Sauvegarde les données Silver au format Parquet."""
    import pyarrow as pa
    import pyarrow.parquet as pq

    silver_path = Path(data_lake_root) / 'silver'

    # Offres nettoyées
```

```
    chemin_offres = silver_path / 'offres_clean' / 'offres_
clean.parquet'
    chemin_offres.parent.mkdir(parents=True, exist_ok=True)
    df_offres.to_parquet(chemin_offres, index=False, compre
ssion='snappy')
    print(f"[SILVER] offres_clean.parquet sauvegardé ({chem
in_offres.stat().st_size // 1024} Ko)")

    # Compétences extraites
    chemin_comp = silver_path / 'competences_extraites' /
'competences.parquet'
    chemin_comp.parent.mkdir(parents=True, exist_ok=True)
    df_competences.to_parquet(chemin_comp, index=False, com
pression='snappy')
    print(f"[SILVER] competences.parquet sauvegardé ({chemi
n_comp.stat().st_size // 1024} Ko)")
```

2.5 Silver → Gold : Calcul des agrégats analytiques

```
import duckdb

def construire_gold(data_lake_root: str):
    """
    Construit toutes les tables Gold depuis les données Sil
ver.
    Utilise DuckDB pour les requêtes SQL directement sur le
s fichiers Parquet.
    """
    silver_offres  = f"{data_lake_root}/silver/offres_clea
n/offres_clean.parquet"
    silver_comp    = f"{data_lake_root}/silver/competences_
extraites/competences.parquet"
    gold_path      = Path(data_lake_root) / 'gold'
    gold_path.mkdir(parents=True, exist_ok=True)

    con = duckdb.connect()

    # ── Table Gold 1 : Top compétences par profil ────────
```

```
────────────────────
    print("[GOLD] Construction top_competences...")
    df_top_comp = con.execute(f"""
        SELECT
            profil,
            famille,
            competence,
            COUNT(DISTINCT id_offre)                    AS
nb_offres_mentionnent,
            ROUND(COUNT(DISTINCT id_offre) * 100.0 /
                (SELECT COUNT(DISTINCT id_offre) FROM '{sil
ver_offres}'), 2)
                                                        AS
pct_offres_total,
            RANK() OVER (
                PARTITION BY profil
                ORDER BY COUNT(DISTINCT id_offre) DESC
            )                                           AS
rang_dans_profil
        FROM '{silver_comp}'
        WHERE competence != 'non_détecté'
        GROUP BY profil, famille, competence
        ORDER BY profil, rang_dans_profil
    """).df()
    df_top_comp.to_parquet(gold_path / 'top_competences.par
quet', index=False)

    # ── Table Gold 2 : Salaires par profil et ville ──────
────────────────────
    print("[GOLD] Construction salaires_par_profil...")
    df_salaires = con.execute(f"""
        SELECT
            profil_normalise        AS profil,
            ville_std               AS ville,
            type_contrat_std        AS type_contrat,
            COUNT(*)                AS nb_offres,
            COUNT(*) FILTER (WHERE salaire_connu)
                                    AS nb_offres_avec_salai
```

```
re,
            ROUND(MEDIAN(salaire_median_mad) FILTER (WHERE
salaire_connu), 0)
                                    AS salaire_median_mad,
            ROUND(AVG(salaire_median_mad) FILTER (WHERE sal
aire_connu), 0)
                                    AS salaire_moyen_mad,
            ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP
                (ORDER BY salaire_median_mad) FILTER (WHERE
salaire_connu), 0)
                                    AS salaire_q1_mad,
            ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP
                (ORDER BY salaire_median_mad) FILTER (WHERE
salaire_connu), 0)
                                    AS salaire_q3_mad,
            ROUND(MIN(salaire_min_mad) FILTER (WHERE salair
e_connu), 0)
                                    AS salaire_min_observe,
            ROUND(MAX(salaire_max_mad) FILTER (WHERE salair
e_connu), 0)
                                    AS salaire_max_observe
        FROM '{silver_offres}'
        GROUP BY profil_normalise, ville_std, type_contrat_
std
        HAVING COUNT(*) >= 5    -- minimum 5 offres pour fi
abilité statistique
        ORDER BY nb_offres DESC
    """).df()
    df_salaires.to_parquet(gold_path / 'salaires_par_profi
l.parquet', index=False)

    # ── Table Gold 3 : Volume d'offres par ville et profil
──────────────────
    print("[GOLD] Construction offres_par_ville...")
    df_villes = con.execute(f"""
        SELECT
            ville_std                           AS ville,
            region_admin,
```

```
            profil_normalise                    AS profil,
            annee,
            mois,
            COUNT(*)                            AS nb_offre
s,
            COUNT(*) FILTER (WHERE teletravail ILIKE '%télé
travail%'
                             OR teletravail ILIKE '%remot
e%'
                             OR teletravail ILIKE '%hybrid
e%')
                                                AS nb_offre
s_remote,
            ROUND(COUNT(*) FILTER (WHERE teletravail ILIKE
'%télétravail%'
                             OR teletravail ILIKE '%remot
e%'
                             OR teletravail ILIKE '%hybrid
e%') * 100.0
                  / NULLIF(COUNT(*), 0), 1)     AS pct_remo
te
        FROM '{silver_offres}'
        GROUP BY ville_std, region_admin, profil_normalise,
annee, mois
        ORDER BY nb_offres DESC
    """).df()
    df_villes.to_parquet(gold_path / 'offres_par_ville.parq
uet', index=False)

    # ── Table Gold 4 : Entreprises les plus recruteurs ──
──────────────────
    print("[GOLD] Construction entreprises_recruteurs...")
    df_entreprises = con.execute(f"""
        SELECT
            entreprise,
            ville_std                               AS vill
e,
            COUNT(*)                                AS nb_o
```

```
ffres_publiees,
            COUNT(DISTINCT profil_normalise)        AS nb_p
rofils_differents,
            ROUND(AVG(salaire_median_mad) FILTER (WHERE sal
aire_connu), 0)
                                                    AS sala
ire_moyen_propose,
            ARRAY_AGG(DISTINCT profil_normalise
                      ORDER BY profil_normalise)    AS prof
ils_recrutes,
            MIN(date_publication)                   AS prem
iere_offre,
            MAX(date_publication)                   AS dern
iere_offre
        FROM '{silver_offres}'
        WHERE entreprise IS NOT NULL
          AND entreprise != ''
        GROUP BY entreprise, ville_std
        HAVING COUNT(*) >= 3
        ORDER BY nb_offres_publiees DESC
        LIMIT 100
    """).df()
    df_entreprises.to_parquet(gold_path / 'entreprises_recr
uteurs.parquet', index=False)

    # ── Table Gold 5 : Tendances mensuelles ──────────────
──────────────────
    print("[GOLD] Construction tendances_mensuelles...")
    df_tendances = con.execute(f"""
        SELECT
            annee,
            mois,
            profil_normalise                        AS prof
il,
            COUNT(*)                                AS nb_o
ffres,
            ROUND(AVG(salaire_median_mad) FILTER (WHERE sal
aire_connu), 0)
```

```
                                                        AS sala
ire_moyen_mois,
            -- Évolution vs mois précédent
            LAG(COUNT(*)) OVER (
                PARTITION BY profil_normalise
                ORDER BY annee, mois
            )                                           AS nb_o
ffres_mois_precedent
        FROM '{silver_offres}'
        GROUP BY annee, mois, profil_normalise
        ORDER BY profil_normalise, annee, mois
    """).df()
    df_tendances.to_parquet(gold_path / 'tendances_mensuell
es.parquet', index=False)

    con.close()
    print(f"[GOLD] 5 tables Gold construites dans{gold_pat
h}")
```

Livrable Étape 2

- Code Python complet et fonctionnel dans un dépôt Git
- Fichier `README.md` avec instructions pour reproduire le pipeline
- Rapport de traitement `rapport_pipeline.md` documentant pour chaque transformation :
  - La règle appliquée
  - Le nombre de lignes avant / après
  - Les cas limites rencontrés et leur traitement

Étape 3 — Analyse du marché (25%)

Répondre aux **5 questions analytiques** suivantes avec DuckDB. Pour chaque question : la requête SQL commentée, le résultat sous forme de tableau, et une **interprétation** en langage naturel.

Question 1 — Quelles compétences sont les plus demandées au Maroc en IT ?

```sql
-- Connexion DuckDB aux fichiers Gold
-- duckdb.connect()

-- Top 20 compétences toutes offres confondues
SELECT
    famille,
    competence,
    nb_offres_mentionnent,
    pct_offres_total,
    rang_dans_profil
FROM read_parquet('data_lake_mexora_rh/gold/top_competence
s.parquet')
WHERE profil = 'tous'     -- agrégat global si disponible
ORDER BY nb_offres_mentionnent DESC
LIMIT 20;

-- Top 5 compétences par profil data
SELECT
    profil,
    famille,
    competence,
    nb_offres_mentionnent,
    rang_dans_profil
FROM read_parquet('data_lake_mexora_rh/gold/top_competence
s.parquet')
WHERE profil IN ('Data Engineer', 'Data Analyst', 'Data Sci
entist')
  AND rang_dans_profil <= 5
ORDER BY profil, rang_dans_profil;
```

Format de réponse attendu :
> Tableau des résultats + paragraphe d'interprétation de 5 à 10 lignes.
> Exemple : *"Python est la compétence la plus demandée avec 68% des offres IT, suivi de SQL (54%) et JavaScript (47%). Parmi les profils data, Python domine dans les trois catégories, mais les Data Engineers se distinguent par une forte demande sur Spark (38%) et Airflow (22%), absents des offres Data Analyst."*

Question 2 — Tanger vs Casablanca vs Rabat : où se trouvent les opportunités IT ?

```
-- Comparaison des 3 principales villes IT
SELECT
    ville,
    profil,
    nb_offres,
    nb_offres_remote,
    pct_remote,
    -- Rang de la ville pour ce profil
    RANK() OVER (PARTITION BY profil ORDER BY nb_offres DES
C) AS rang_ville
FROM read_parquet('data_lake_mexora_rh/gold/offres_par_vill
e.parquet')
WHERE ville IN ('Casablanca', 'Rabat', 'Tanger', 'Marrakec
h', 'Fès')
ORDER BY profil, rang_ville;

-- Focus Tanger : opportunités spécifiques
SELECT
    profil,
    nb_offres,
    pct_remote,
    -- Tanger vs Casablanca (ratio)
    ROUND(nb_offres * 100.0 /
        NULLIF(SUM(nb_offres) FILTER (WHERE ville = 'Casabl
anca')
               OVER (PARTITION BY profil), 0), 1) AS pct_vs
_casa
FROM read_parquet('data_lake_mexora_rh/gold/offres_par_vill
e.parquet')
WHERE ville = 'Tanger'
ORDER BY nb_offres DESC;
```

Interprétation attendue : Cette question est stratégique pour Mexora basée à Tanger. Si le marché local est peu fourni pour certains profils, cela justifie soit

de recruter à Casablanca/Rabat en remote, soit d'offrir des packages plus attractifs pour attirer des talents depuis d'autres villes.

Question 3 — Quel est le salaire médian par profil IT au Maroc ?

```
-- Salaires médians par profil (toutes villes)
SELECT
    profil,
    SUM(nb_offres)                                  AS nb_o
ffres_total,
    SUM(nb_offres_avec_salaire)                     AS nb_a
vec_salaire,
    ROUND(SUM(nb_offres_avec_salaire) * 100.0
        / NULLIF(SUM(nb_offres), 0), 1)             AS pct_
salaire_communique,
    MEDIAN(salaire_median_mad)                      AS sala
ire_median_mad,
    MIN(salaire_min_observe)                        AS sala
ire_plancher,
    MAX(salaire_max_observe)                        AS sala
ire_plafond
FROM read_parquet('data_lake_mexora_rh/gold/salaires_par_pr
ofil.parquet')
GROUP BY profil
ORDER BY salaire_median_mad DESC NULLS LAST;

-- Salaires à Tanger spécifiquement (pour les recommandatio
ns Mexora)
SELECT
    profil,
    nb_offres,
    salaire_median_mad,
    salaire_q1_mad,
    salaire_q3_mad,
    -- Différence vs médiane nationale (référence)
    ROUND(salaire_median_mad - MEDIAN(salaire_median_mad)
        OVER (PARTITION BY profil), 0) AS ecart_mediane_nat
ionale
```

```
FROM read_parquet('data_lake_mexora_rh/gold/salaires_par_pr
ofil.parquet')
WHERE ville = 'Tanger'
  AND nb_offres >= 5
ORDER BY salaire_median_mad DESC;
```

Interprétation attendue : Tableau clair des fourchettes de salaires par profil. Les recommandations pour Mexora doivent préciser si les salaires à Tanger sont inférieurs, équivalents ou supérieurs à la médiane nationale, et ce que cela implique pour la stratégie de recrutement.

Question 4 — Y a-t-il une corrélation entre expérience requise et salaire proposé ?

```
-- Relation expérience / salaire par profil
SELECT
    profil_normalise                                    AS
profil,
    CASE
        WHEN experience_min_ans = 0          THEN '0 — Débu
tant'
        WHEN experience_min_ans BETWEEN 1 AND 2 THEN '1-2 a
ns'
        WHEN experience_min_ans BETWEEN 3 AND 4 THEN '3-4 a
ns'
        WHEN experience_min_ans BETWEEN 5 AND 7 THEN '5-7 a
ns'
        WHEN experience_min_ans >= 8         THEN '8+ ans S
enior'
        ELSE 'Non précisé'
    END                                                 AS
tranche_experience,
    COUNT(*)                                            AS
nb_offres,
    ROUND(MEDIAN(salaire_median_mad)
        FILTER (WHERE salaire_connu), 0)               AS s
alaire_median,
    -- Corrélation de Pearson (DuckDB natif)
```

```
    ROUND(CORR(experience_min_ans, salaire_median_mad)
        FILTER (WHERE salaire_connu AND experience_min_ans
IS NOT NULL), 3)
        OVER (PARTITION BY profil_normalise)          AS c
orrelation_pearson
FROM read_parquet('data_lake_mexora_rh/silver/offres_clean/
offres_clean.parquet')
GROUP BY profil_normalise, tranche_experience, experience_m
in_ans, salaire_connu,
         salaire_median_mad
ORDER BY profil, tranche_experience;
```

Interprétation attendue : Commenter la valeur du coefficient de corrélation de Pearson obtenu (entre -1 et 1). Une corrélation > 0.6 indique une relation forte. Préciser si la progression salariale est linéaire ou si elle présente des paliers (par exemple, le saut entre 3-4 ans et 5+ ans).

Question 5 — Quelles entreprises recrutent le plus ? Qui sont les concurrents de Mexora sur le marché du talent ?

```
-- Top 20 entreprises recruteurs
SELECT
    entreprise,
    ville,
    nb_offres_publiees,
    nb_profils_differents,
    salaire_moyen_propose,
    -- Classement par volume de recrutement
    RANK() OVER (ORDER BY nb_offres_publiees DESC) AS rang_
recruteur
FROM read_parquet('data_lake_mexora_rh/gold/entreprises_rec
ruteurs.parquet')
ORDER BY nb_offres_publiees DESC
LIMIT 20;

-- Focus : entreprises recrutant des profils data à Tanger
-- (concurrents directs de Mexora)
SELECT
```

```
    entreprise,
    nb_offres_publiees,
    profils_recrutes,
    salaire_moyen_propose,
    -- Mexora devra aligner ou dépasser ce salaire pour att
irer
    CASE
        WHEN salaire_moyen_propose > 20000 THEN 'Compétiteu
r fort'
        WHEN salaire_moyen_propose > 12000 THEN 'Compétiteu
r moyen'
        ELSE 'Compétiteur faible'
    END AS niveau_competition
FROM read_parquet('data_lake_mexora_rh/gold/entreprises_rec
ruteurs.parquet')
WHERE ville = 'Tanger'
  AND array_contains(profils_recrutes, 'Data Engineer')
     OR array_contains(profils_recrutes, 'Data Analyst')
ORDER BY salaire_moyen_propose DESC NULLS LAST;
```

Livrable Étape 3

- Notebook Python/Jupyter `analyse_marche_it_maroc.ipynb` contenant :
  - Chaque requête DuckDB avec résultat
  - Visualisation (matplotlib / seaborn / plotly) pour chaque question
  - Interprétation textuelle sous chaque graphique

Étape 4 — Rapport analytique et recommandations (15%)

4.1 Structure du rapport (10 pages maximum)

Le rapport doit être rédigé **à destination du DRH de Mexora** — non technique, orienté décision. Il doit se lire sans avoir besoin de voir le code.

```
RAPPORT : Analyse du Marché de l'Emploi IT au Maroc
Mexora RH Intelligence – Novembre 2024
────────────────────────────────────────────────────


1. RÉSUMÉ EXÉCUTIF (1 page)
```

```
   → 5 chiffres clés
   → 3 recommandations prioritaires pour Mexora
   → Horizon de mise en œuvre

2. MÉTHODOLOGIE (1 page)
   → Sources des données (Rekrute, MarocAnnonce, LinkedIn)
   → Période couverte
   → Limites et biais identifiés
   → Architecture Data Lake utilisée

3. ÉTAT DU MARCHÉ IT AU MAROC (3 pages)
   → Volume d'offres par profil et tendance 2023-2024
   → Répartition géographique (Casablanca vs Rabat vs Tanger)
   → Montée du télétravail et hybride
   → Types de contrats dominants (CDI vs Freelance vs CDD)

4. COMPÉTENCES LES PLUS DEMANDÉES (2 pages)
   → Top 10 compétences toutes offres confondues
   → Compétences spécifiques aux profils data
   → Émergence de nouvelles compétences (2024 vs 2023)

5. ANALYSE SALARIALE (2 pages)
   → Salaires médians par profil au Maroc
   → Comparaison Tanger vs médiane nationale
   → Corrélation expérience / salaire
   → Entreprises les mieux rémunératrices

6. RECOMMANDATIONS POUR MEXORA (1 page)
   → Profils prioritaires à recruter (justification)
   → Fourchettes salariales recommandées par poste
   → Tanger : avantages et défis du recrutement local
   → Stratégie de fidélisation (compétences rares à former en interne)
```

4.2 Dashboard de synthèse

Un dashboard visuel (Power BI, Metabase ou Matplotlib/Plotly en notebook) avec **4 visualisations minimum** :

1. **Carte du Maroc** — volume d'offres IT par ville (carte choroplèthe ou bubbles)
2. **Top 15 compétences** — barres horizontales triées, colorées par famille
3. **Boxplot salaires** — distribution des salaires par profil (médiane, Q1, Q3, min/max)
4. **Évolution mensuelle** — courbe du nombre d'offres Data Engineer / Data Analyst / Data Scientist depuis janvier 2023

📊 Grille d'évaluation

Étape	Critères	Points
Étape 1 — Architecture	Justification des formats Bronze/Silver/Gold	8
	Réponses aux questions de conception	7
	Schéma d'architecture clair	5
Étape 2 — Pipeline Python	Ingestion Bronze correcte	5
	Nettoyage Silver (normalisation villes, salaires, expérience)	10
	Extraction de compétences depuis texte libre	15
	Construction des tables Gold	10
Étape 3 — Analyse DuckDB	5 requêtes correctes et documentées	15
	Qualité des interprétations	10
Étape 4 — Rapport	Structure et lisibilité pour non-techniciens	5
	Pertinence des recommandations pour Mexora	5
	Qualité du dashboard (4 visualisations)	5
TOTAL		**100**

📅 Planning recommandé

Semaine	Objectif
Semaine 1	Étape 1 complète + mise en place de la structure du Data Lake + Bronze ingestion
Semaine 2	Étape 2 — pipeline Silver complet (nettoyage + extraction compétences)
Semaine 3	Étape 2 Gold + Étape 3 requêtes DuckDB + visualisations
Semaine 4	Étape 4 — rédaction rapport + dashboard + relecture finale

🛠️ Stack technique recommandé

Outil	Usage	Installation
Python 3.11+	Pipeline de traitement	python.org

Outil	Usage	Installation
pandas	Manipulation de données	`pip install pandas`
pyarrow	Lecture/écriture Parquet	`pip install pyarrow`
DuckDB	Requêtes SQL sur Parquet	`pip install duckdb`
re (stdlib)	Extraction de compétences par regex	inclus Python
matplotlib / seaborn	Visualisations dans le notebook	`pip install matplotlib seaborn`
plotly	Visualisations interactives (option)	`pip install plotly`
Jupyter Notebook	Notebook d'analyse	`pip install jupyter`
Git + GitHub	Versioning du code	git-scm.com

📋 Livrables finaux récapitulatifs

#	Livrable	Format
L1	Document de conception Data Lake	PDF (2 pages max)
L2	Schéma d'architecture	Image (PNG/PDF)
L3	Code Python pipeline complet	Dépôt Git
L4	Rapport pipeline (transformations documentées)	Markdown
L5	Notebook d'analyse DuckDB	`.ipynb`
L6	Rapport analytique marché IT Maroc	PDF (10 pages max)
L7	Dashboard de synthèse (4 visualisations)	PDF ou lien partagé